



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Smart Parkinglot

Advanced IoT and Intelligent Cyber-Physical Systems
Masterstudiengang Informatik
Sommersemester 2026

Studienarbeit

Vorgelegt von: Michael Amann
Matrikelnummer: 3281039

Teammitglieder: Oliver Masur
Jakob Frohwein
Maximilian Strauss
Philipp Zausinger

Abgabedatum: 19.07.2026

Inhaltsverzeichnis

I	Abbildungsverzeichnis	II
1	Projektbeschreibung	1
1.1	Zielsetzung	1
1.2	Projektarchitektur	1
1.3	Fokus dieser Studienarbeit	2
2	Parkplatzbelegung	3
2.1	Architektur	3
2.2	Hardware und Verdrahtung	4
2.3	Interne Logik und Kommunikation	5
2.3.1	Erkennung eines Objekts	5
2.3.2	Steuerung der Parkplatzbelegung	6
2.3.3	LED-Steuerung	7
2.4	Visualisierung	7
3	Diskussion	8
4	Literaturverzeichnis	9
5	KI Klausel	9

I Abbildungsverzeichnis

1	Architektur des Smart-Parking-IoT-Systems	2
2	Architektur des Teilsystems “Parkplatzbelegung”	3
3	Skematische Verdrahtung des Ultraschallsensors HC-SR04 mit dem ESP32	5
4	Ausschnitt des Node-Red Flows zur zentralen Steuerung	7
5	Dashboard zur Parkplatzbelegung	7

1 Projektbeschreibung

Im Rahmen des Masterkurses „Advanced IoT and Intelligent Cyber-Physical Systems“ wurde als Abschlussprojekt ein Smart-Parking-System in Form eines Modellparkhauses entwickelt. Das System verbindet verschiedene Hardware- und Softwarekomponenten zu einer gemeinsamen IoT-Anwendung. Dabei werden grundlegende Abläufe eines automatisierten Parkhauses abgespeichert, verwaltet und schlussendlich visualisiert. Die Zielsetzung sowie der technische Aufbau des Systems werden in den folgenden Abschnitten näher beschrieben, bevor der Fokus dieser Arbeit im Detail dargestellt wird.

1.1 Zielsetzung

Ziel des Projekts ist es, ein funktionsfähiges IoT-basiertes Parkhaussystem zu entwickeln, das typische Prozesse eines modernen Parkhauses realitätsnah darstellt. Dieses Projekt dient als Prototyp, um die wichtigsten Grundfunktionen eines automatisierten Parkhauses abzubilden. Dies umfasst das Erkennen von Kennzeichen und Parkplatzbelegungen, das Verwalten von Parkvorgängen sowie das Steuern der Schranken und der Etagenbeleuchtung.

Der Fokus liegt dabei nicht auf der Entwicklung eines fertigen Produktivsystems, sondern vielmehr auf der technischen Umsetzung und Integration der einzelnen Teilsysteme.

1.2 Projektarchitektur

Die Architektur des Smart-Parking-IoT-Systems ist modular aufgebaut und besteht aus einer zentralen Steuerung sowie mehreren dezentralen IoT-Komponenten. Der zentrale Raspberry Pi bildet den Kern des Systems. Auf ihm laufen der MQTT-Broker, das Node-RED-Backend, die Datenbank sowie die REST-API.

Die Raspberry Pis an der Ein- und Ausfahrt übernehmen die Kennzeichenerkennung mittels eines YOLO-Modells. Nach erfolgreicher Erkennung wird das Kennzeichen über MQTT an das Backend gesendet. Am Eingang wird daraufhin ein Parkvorgang erstellt und die Schranke geöffnet. An der Ausfahrt wird geprüft, ob der Parkvorgang bezahlt und das Ausfahrtszeitfenster gültig ist. Zusätzlich kann die Ausfahrt durch einen Administrator per RFID freigegeben werden.

Innerhalb des Parkhauses erfassen ESP32-Mikrocontroller die Parkplatzbelegung und Bewegungen. Basierend auf diesen Daten wird auch die Beleuchtung gesteuert. Die Kommunikation mit dem Backend findet über MQTT statt.

Die Dashboards greifen über die REST-API auf das Backend zu. Abbildung 1 zeigt die wichtigsten Komponenten sowie deren Kommunikationswege.

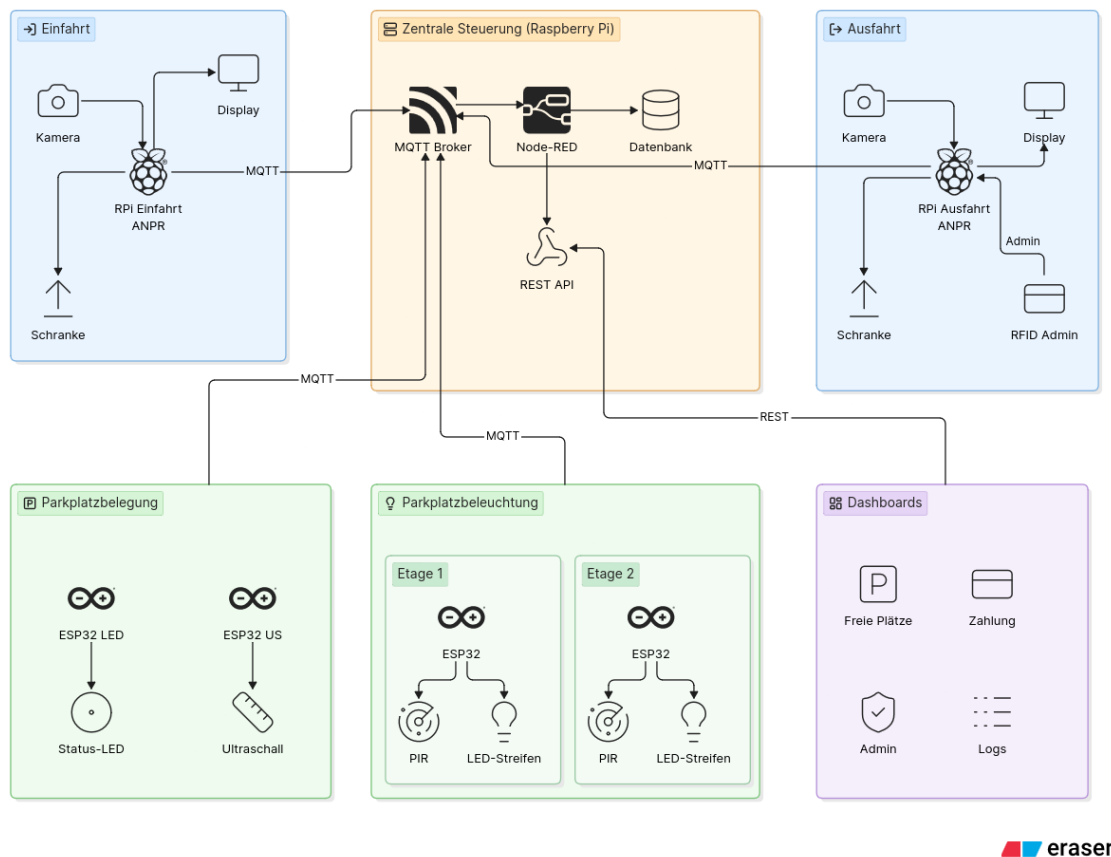


Abbildung 1: Architektur des Smart-Parking-IoT-Systems

1.3 Fokus dieser Studienarbeit

Diese Studienarbeit fokussiert sich auf das Thema Parkplatzbelegung. Dabei handelt es sich um das komplette Teilsystem, welches sich mit der Belegung der einzelnen Parkplätze befasst. Konkret umfasst dieses Modul zwei Node-RED-Flows zur Steuerung des Systems, eine Messkomponente und ein LED-Modul zur physischen Darstellung. Obwohl das Parkhausmodell mehrere Parkplätze beinhaltet, wurde dieses Teilsystem zu Demonstrationszwecken einmal implementiert, kann allerdings beliebig skaliert werden.

Im Folgenden wird dieses Teilsystem detailliert präsentiert und es werden gewisse Entwicklungsentscheidungen näher erläutert. Abgeschlossen wird die Studienarbeit mit einer Diskussion sowie weiteren nichtfunktionalen Anforderungen wie Sicherheit dieses Systems.

2 Parkplatzbelegung

Wie bereits beschrieben, befasst sich dieses Teilsystem mit der Erkennung belegter oder verfügbarer Parkplätze.

Grundsätzlich folgt dieses Modul der kanonischen CPS-Kontrollschleife. Demnach werden zunächst physische Prozesse mittels Sensoren gemessen. Konkret handelt es sich hierbei um die Distanzen, welche mit einem Ultraschallsensor erfasst werden. Diese Daten werden anschließend direkt innerhalb des *Embedded Devices* anhand eines Algorithmus bewertet oder in einem Zwischenschritt an ein anderes Edge-System weitergeleitet. Auf Basis der Entscheidung wird ein Aktuator angesteuert, um einen physischen Prozess auszulösen. Umgesetzt wird dies durch die Steuerung einer LED, die den aktuellen Status eines Parkplatzes darstellt. Diese CPS-Schleife läuft durchgehend und liefert so die aktuelle Parkplatzbelegung in Echtzeit.

2.1 Architektur

Dieses Teilsystem ist so aufgebaut, dass es unabhängig von den restlichen modularen Komponenten des Gesamtsystems laufen kann. Außerdem ist das Modul in sich modular aufgebaut, was das System flexibel und erweiterbar macht. In Abbildung 2 ist die Architektur dargestellt.

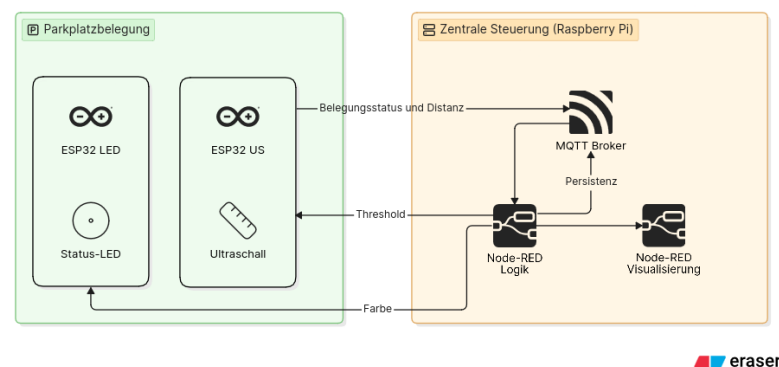


Abbildung 2: Architektur des Teilsystems “Parkplatzbelegung”

Eine Instanz dieses Teilsystems beinhaltet zwei Hardwaremodule sowie ein Logikmodul. Unter einer Instanz versteht man die konkrete Implementierung dieses Systems. Da die Parkplatzbelegung für jeden Parkplatz individuell stattfinden muss, werden auch entsprechend viele Instanzen benötigt. In Abbildung 2 sind die zwei Hardwarekomponenten im Bereich **Parkplatzbelegung** dargestellt. Die interne Logik läuft auf den *Embedded Devices* sowie innerhalb des Node-RED-Flows **Logik**.

Zusätzlich gibt es einen separaten Node-RED-Flow **Visualisierung**, welcher für die

visuelle Präsentation der Parkplatzbelegung zuständig ist. Dieser muss allerdings nicht für jede Instanz neu erstellt werden, sondern dient als Schnittstelle für alle Instanzen dieses Teilsystems. Eine detaillierte Erläuterung dieses Teilbereichs folgt in Kapitel 2.4.

Für die Kommunikation der einzelnen Komponenten wird ein Raspberry Pi als zentrale Steuerungseinheit verwendet. Dieser wird nicht nur in diesem Teilbereich, sondern im Gesamtsystem als Kommunikationsschnittstelle verwendet. Auf dem Raspberry Pi läuft die Anwendung *Mosquitto*, die somit als MQTT-Broker dient. Alle Datenflüsse aus Abbildung 2 finden über MQTT statt. Entsprechend werden für die Kommunikation spezifische Topics *published* oder *subscribed*, die in Kapitel 2.3 genauer dargestellt werden.

2.2 Hardware und Verdrahtung

Für dieses System werden zwei *Embedded Devices* in Form von zwei ESP32, ein LED-Shield als Aktuator und ein HC-SR04-Ultraschallsensor verwendet. Das LED-Shield wurde so ausgewählt, dass es direkt auf den ESP32 *gemountet* werden konnte. Dementsprechend wird keine zusätzliche Verdrahtung benötigt.

Der Ultraschallsensor benötigt allerdings einen Spannungsteiler, um den ESP32 nicht zu beschädigen. Grundsätzlich funktioniert der HC-SR04, indem Ultraschallsignale verschickt und empfangen werden. Anhand der Dauer dieses Vorgangs kann anschließend der Abstand zu einem Objekt vor dem Sensor bestimmt werden. Der HC-SR04 benötigt 5 V als *VCC*, um ordnungsgemäß zu operieren. Kritisch wird die Verwendung dieses Sensors beim Verbinden des Echo-Pins mit dem ESP32. ESP32-Mikrocontroller können an GPIO-Pins nur Signale mit maximal 3,6 V verarbeiten [1]. Der Ultraschallsensor schickt allerdings 5-V-Signale über den Echo-Pin [2]. Deshalb muss ein zusätzlicher Spannungsteiler zwischen den Ultraschallsensor und den ESP32 gesetzt werden. Ohne zu genau in die Elektrotechnik einzutauchen, erlaubt es dieser Teiler, das Signal auf 3,3 V zu reduzieren. Dies schützt den ESP32 vor Schäden durch zu hohe Spannung.

Die schematische Verdrahtung des ESP32 mit dem HC-SR04 ist in Abbildung 3 dargestellt.

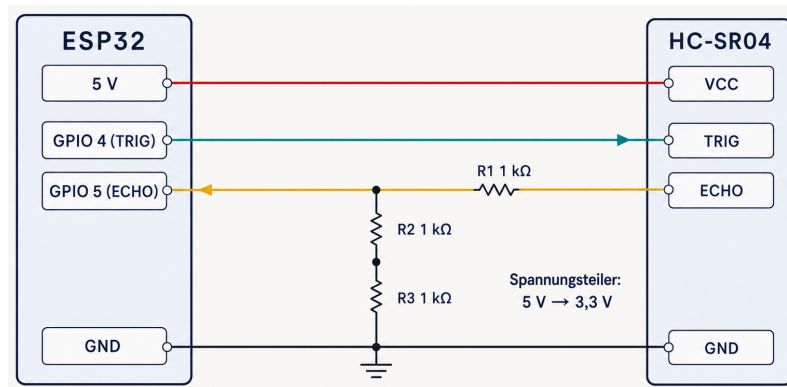


Abbildung 3: Skematische Verdrahtung des Ultraschallsensors HC-SR04 mit dem ESP32

Es ist zu erkennen, dass die Resistoren R1, R2 und R3 so platziert wurden, um die Spannung auf 3,3 V zu senken. Die Prinzipien und Funktionsweisen des Spannungsteilers werden hier nicht genauer spezifiziert.

2.3 Interne Logik und Kommunikation

Die interne Logik einer Instanz des Teilsystems findet getrennt in zwei Teilbereichen statt. Dabei wird zwischen der Logik zur physischen Erkennung eines Objekts und der Logik zur Steuerung des Teilsystems unterschieden. Diese sowie die Kommunikation zwischen den einzelnen Einheiten werden im Folgenden genauer beschrieben.

2.3.1 Erkennung eines Objekts

Die Erkennung, ob sich ein Objekt auf dem Parkplatz befindet, läuft auf dem ESP32, welcher mit dem Ultraschallsensor verbunden ist. Da der Ultraschallsensor zumindest im Prototyp sehr *noisy* Daten geliefert hat, wird mittels des Medians versucht, diese zu stabilisieren.

Demnach werden zunächst fünf Sensorwerte gesammelt und daraus der Median ermittelt. Für die darauffolgenden Schritte wird nun dieser ermittelte Wert verwendet. Dies bezweckt, dass vereinzelt falsche Sensorwerte nicht direkt im Entscheidungsprozess berücksichtigt werden.

Nach dem Sammeln der Daten wird anhand eines *Thresholds* bestimmt, ob sich ein Objekt auf dem Parkplatz befindet oder nicht. Dabei gilt:

$$\text{Result} = \begin{cases} \text{TRUE}, & \text{falls } \text{MED}(\text{Eingabedaten}) < \text{Threshold}, \\ \text{FALSE}, & \text{sonst.} \end{cases}$$

Sprich: Ein Objekt wird erkannt, wenn die Distanz geringer als der *Threshold* ist. Diese Information wird an den Node-RED-Flow **Logik** aus Abbildung 2 gesendet, indem sie auf einem bestimmten Topic *published* wird, das von diesem Flow *subscribed* wird.

Zusätzlich ist es möglich, diesen *Threshold* während des Betriebs zu ändern, indem ein neuer Wert auf dem dafür vorgesehenen Topic *published* wird. Dieses Topic wird vom Sensor *subscribed*, der den bereitgestellten Wert übernimmt.

2.3.2 Steuerung der Parkplatzbelegung

Der Node-RED-Flow **Logik** aus Abbildung 2 stellt das zweite logische Modul des Systems dar. Die Informationen der Objekterkennung werden über ein *MQTT-IN* erfasst und anschließend in einem Array A mit fester Länge nach dem *Sliding-Window*-Prinzip gespeichert. Demnach wird das Array A von links befüllt, bis dieses voll ist (*push*). Wenn dieses komplett befüllt ist, wird der älteste Sensorwert bei jedem darauffolgenden Einfügen gelöscht.

Auf Basis dieses Arrays A findet die tatsächliche Entscheidung statt, wie das System reagieren soll.

$$z_{\text{neu}} = \begin{cases} \text{belegt,} & \text{falls } \frac{\#\{a_i \in A \mid a_i = \text{TRUE}\}}{|A|} \geq 0,8, \\ \text{frei,} & \text{falls } \frac{\#\{a_i \in A \mid a_i = \text{FALSE}\}}{|A|} \geq 0,8, \\ z_{\text{alt}}, & \text{sonst.} \end{cases}$$

Zusammengefasst bedeutet dies, dass der Parkplatz erst als “Belegt” interpretiert wird, wenn 80 % der Sensorwerte dieses Resultat bestätigen.

Dadurch wird gewährleistet, dass vereinzelte *False Positives* oder temporäre Objekte, beispielsweise eine Person, die unter dem Sensor vorbeigeht, nicht direkt als belegter Parkplatz gewertet werden.

Abhängig von dieser Entscheidung werden folgende Aktionen gestartet:

- Der ESP32 mit der LED wird angesprochen, um die Farbe zu aktualisieren.
- Dem Node-RED-Flow **Visualisierung** wird der aktuelle Status mitgeteilt.
- Falls es zu einer Statusänderung (z. B. Belegt” → Verfügbar”) kommt, wird eine Nachricht zur persistenten Speicherung an das Backend gesendet.

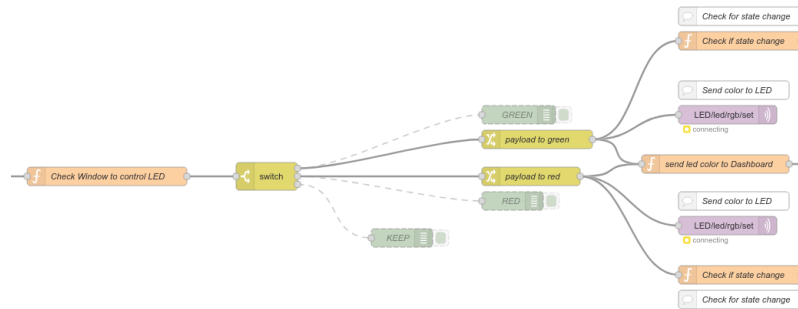


Abbildung 4: Ausschnitt des Node-Red Flows zur zentralen Steuerung

Ein Ausschnitt des Node-RED-Flows **Logik** ist in Abbildung 4 zu sehen. Am rechten Rand ist zu erkennen, wie zwei der drei genannten Aktionen durchgeführt werden. Abgeschnitten sind die Speicherung des aktuellen Zustands und das **MQTT-OUT** bei Zustandswechseln.

2.3.3 LED-Steuerung

Für die Steuerung der LED gibt es keine eigenständige Logik. Diese wurde identisch aus den Übungsaufgaben mit IoTempower übernommen und ebenfalls mittels Io-Tempower *geflasht*. Entsprechend abonniert der ESP32 mit dem LED-Shield fest definierte Topics und ändert dadurch die Farbe der LED.

2.4 Visualisierung

Das Diagramm für die Parkplatzbelegung basiert auf der Node-RED-Library “Flow-Fuse”. Das Dashboard wird in Abbildung 5 dargestellt.

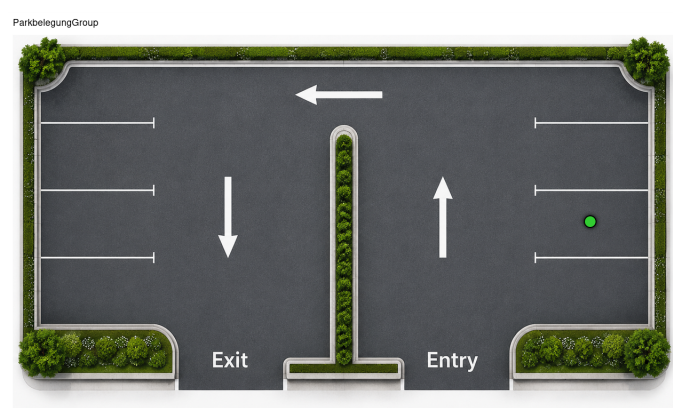


Abbildung 5: Dashboard zur Parkplatzbelegung

Für die Erstellung des Dashboards wurde ein *UI-Template*-Widget genutzt, wodurch ein Dashboard basierend auf HTML und CSS generiert werden konnte. Kommt nun

ein Signal von einem Sensor an, wird die Farbe des Kreises für den jeweiligen Sensor angepasst. In Abbildung 5 ist der Parkplatz beispielsweise noch verfügbar.

3 Diskussion

Da es sich bei diesem Projekt um einen reinen Prototyp handelt, gibt es einige Bereiche, welche kaum oder sogar gar nicht implementiert wurden.

Sicherheit

Das Thema Sicherheit wurde bei der Entwicklung nicht berücksichtigt. Entsprechend gibt es keine Sicherheitsmechanismen, was triviale Cyberangriffe wie das einfache *Sniffing* (Abhören) der Daten erlaubt. Dies wurde so gemacht, weil wir den Fokus bewusst auf die Funktionalität und die Integration der einzelnen Module im Gesamtsystem gesetzt haben.

Dadurch bedarf es einer strengen Sicherheitsanalyse, bevor dieser Prototyp in einem beliebigen Produktivsystem eingesetzt werden sollte.

Probleme bei der Entwicklung

Bei der Entwicklung gab es hauptsächlich Probleme mit der Genauigkeit des Sensors. Da der Prototyp mit kleinen Spielzeugautos getestet wurde, musste der Sensor sehr genaue Messungen liefern, da teilweise nur wenige Zentimeter Differenz bestanden.

Der Ultraschallsensor konnte diese Differenzen tatsächlich erkennen und anhand eines präzise definierten Thresholds hat die Parkplatzbelegung wie gewünscht funktioniert. Leider hat dies allerdings nicht durchgängig funktioniert und es gab viele Messungen, welche nicht akkurat waren.

Aufgrund der begrenzten Zeit konnte das System leider nicht in einer produktionsnahen Umgebung getestet werden. Dieser Vergleich wäre sehr interessant gewesen, vor allem was die Präzision der Distanzmessungen angeht.

Ein weiterer Faktor, welcher bei der Entwicklung gestört hat, war die kurzfristige Änderung der Wi-Fi-Infrastruktur. Mit der Nutzung des Routers und des Switches haben sich auch die Wi-Fi-Zugangsdaten und die IP-Adresse des MQTT-Brokers geändert. Entsprechend mussten alle Geräte im gesamten System neu *geflasht* werden. Zurückblickend würde ich solche großen Entscheidungen wahrscheinlich zu Beginn des Projekts treffen, um den Aufwand sowie auftretende Risiken zu minimieren.

4 Literaturverzeichnis

- [1] „Hardware Design“, Espressif. (), URL: https://docs.espressif.com/projects/esp-faq/en/latest/hardware-related/hardware-design.html?utm_source=chatgpt.com (besucht am 18.07.2026).
- [2] „Connecting HC-SR04 ultrasonic sensor with ESP32“, implrust. (), URL: <https://esp32.implrust.com/ultrasonic/circuit.html#connecting-hc-sr04-ultrasonic-sensor-with-esp32> (besucht am 18.07.2026).

5 KI Klausel

ChatGPT und Eraser.io wurden punktuell zur sprachlichen Überarbeitung einzelner Textstellen sowie zur Erstellung der Architekturdiagramme eingesetzt. Konzeption und Umsetzung der fachlichen Inhalte stammen vom Verfasser. Sämtliche Vorschläge wurden eigenständig geprüft und bei Bedarf angepasst.